

- Project Final Report: Implementation of a Domain-Specific Large Language Model for Banking Customer Service
 - 1. Introduction
 - 2. Tools, Libraries, and Techniques Used
 - Large Language Model (LLM) Layer
 - Embedding and Vector Retrieval Layer (RAG)
 - System Integration & Backend
 - Frontend
 - Guardrails
 - 3. Fine-Tuning Implementation Approach
 - Fine-Tuning Techniques and Pipeline (src/finetune_lora.py):
 - 4. Architecture Overview
 - Data Ingestion pipeline
 - Prompt Augmentation & RAG Flow
 - 5. Usage Instructions and Setup
 - Prerequisites
 - 1. Environment Variables
 - 2. Dependency Installation
 - 3. Build Vector Indexes (Required first time):
 - 4. Running the Complete System

Project Final Report: Implementation of a Domain-Specific Large Language Model for Banking Customer Service

Course: CS-416 Large Language Models **Instructors:** Prof. Dr. Faisal Shafait, Dr. Momina Moetesum **Class:** BESE 13B - Fall'25 **Group Members:**

- Saleha Zainab Fatima (404329)
- Asma Saeed (416741)

Submission: Complete Submission (Final Course Project)

1. Introduction

The NUST Bank AI Customer Service Agent is a Retrieval-Augmented Generation (RAG) system built to enhance banking customer service. It answers customer queries securely and accurately using a structured database of FAQs, policies, and product documents while ensuring guardrails for data privacy and domain specificity.

2. Tools, Libraries, and Techniques Used

Large Language Model (LLM) Layer

- **Model:** `Qwen/Qwen2.5-3B-Instruct`
 - Selected over the initially proposed Llama 3.2-3B due to robust reasoning capabilities, instruction-tuning for system prompt adherence, and successful inference capability via the HuggingFace Inference API.
- **Transformers & Accelerate:** Leveraging HuggingFace libraries to handle model operations safely on hardware.

Embedding and Vector Retrieval Layer (RAG)

- **Embedding Model:** `Alibaba-NLP/gte-large-en-v1.5` (1024-dimensional)
 - During the project lifecycle, we upgraded to `gte-large-en-v1.5` over `all-MiniLM-L6-v2`. As a large English-optimized model with an MTEB score of 65.4, it provides highly precise semantic similarity critical for banking retrieval versus the 56.3 score of MiniLM.
- **Vector Store:** `FAISS` (Facebook AI Similarity Search) & `faiss-cpu`
 - Utilized `IndexFlatIP` (L2-normalized) for exact inner product/cosine similarity search. Exhaustive search meets speed requirements effectively due to the index size.

System Integration & Backend

- **FastAPI:** Async backend handling LLM integration, custom conversation memory, RAG endpoints, and real-time document parsing (`uvicorn`, `pydantic`).
- **LangChain:** Used alongside custom orchestrations for RAG chunk sizes and prompt structuring.

Frontend

- **React & Vite:** A customized, responsive web interface lacking bulky UI frameworks to offer a swift and seamless user experience. Includes markdown parsing and conversational memory UI.

Guardrails

- **Regex Detectors & `better-profanity`:** Lightweight, efficient guardrails handle input and output control.
 - **Input Guardrails:** 16 robust regex-patterns detect Jailbreaks, hypothetical "DAN" directives, and off-topic filtering.
 - **Output Guardrails:** Post-LLM masking of sensitive information (PII) including IBANs, Credit Cards, and CNICs (Pakistani National ID format).
-

3. Fine-Tuning Implementation Approach

To specialize the open-source 3B model entirely for NUST Bank's specific dialogue style without overwhelming computing resources, we perform Parameter-Efficient Fine-Tuning (PEFT) using the **QLoRA** strategy.

Fine-Tuning Techniques and Pipeline (`src/finetune_lora.py`):

1. **Data Preprocessing & ChatML:** The existing bank chunks (FAQs, Data Sheets) are loaded into a HuggingFace `Dataset`. They are reformatted using the model's exact chat template (System + User + Assistant messages) guaranteeing the LLM sees data during training exactly as it's passed during inference.
2. **4-Bit Quantization (BitsAndBytes):** We initialize `BitsAndBytesConfig` using `nf4` (NormalFloat 4-bit) combined with `bfloat16` compute datatypes. This aggressively shrinks memory usage.
3. **LoRA Adapters (`peft`):** Using Low-Rank Adaptation (`LoraConfig`) targeted at the attention modules (`q_proj`, `v_proj`). It operates at dimension `r=16` with

`alpha=32`, freezing the base architecture and updating tiny specific matrices.

4. **SFT Trainer:** The `SFTTrainer` (from the `trl` library) optimizes the process utilizing a memory-efficient `paged_adamw_8bit` optimizer and gradient accumulation to simulate large batch sizes on commodity hardware.
 5. **Adapter Overlay:** The final trained LoRA weights are extremely small (just several MBs) and can be loaded dynamically onto the base model.
-

4. Architecture Overview

Data Ingestion pipeline

Incoming unstructured `.xlsx` sheets and `.json` arrays are sanitized and flattened into uniform text chunks containing contexts, sources, and categories. (See: `src/ingest.py`)

Prompt Augmentation & RAG Flow

When a user asks a question via the Frontend:

1. **Security Pass:** The input is validated against malicious patterns.
 2. **History Prepending:** We prepend the two most recent interactions logically to resolve user coreferences (e.g., "What's the interest rate on *that*?").
 3. **Retrieval Search:** Top-k similar vectors are fetched heavily relying on a two-pass threshold (Strict first, then fallback logic).
 4. **Context Building:** The fetched data is chained tightly containing custom profile parameters, the 4-turn chat history, and retrieved data chunks.
 5. **LLM Inference:** Sent to Qwen2.5-3B instruction model via huggingface.
 6. **Final Output Guardrail:** Information is scrubbed of PII and returned to the client. Real-time metrics (latency, citation sources) are populated.
-

5. Usage Instructions and Setup

Prerequisites

- HuggingFace API key
- Python 3.10+
- Node.js 18+

1. Environment Variables

Create a `.env` file referencing your keys:

```
HF_API_TOKEN=your_huggingface_token
```

2. Dependency Installation

Execute to build the proper virtual environment and dependencies:

```
python -m venv venv
# Activate the environment (Windows: venv\Scripts\activate | macOS/Linux: source venv/bin/activate)
pip install -r requirements.txt
```

3. Build Vector Indexes (Required first time):

Transform knowledge assets inside `Bank Knowledge/` into FAISS embeddings:

```
python src/vector_store.py
```

4. Running the Complete System

Backend Server:

```
uvicorn backend.main:app --reload --port 8000
```

This deploys the backend system across `localhost:8000` with automated Docs at `/docs`.

Frontend React Server: Open a new terminal session, then install modules and initialize Vite:

```
cd frontend  
npm install  
npm run dev
```

Navigate to <http://localhost:5173> to access the chat UI! New knowledge documents or data sources can be uploaded instantly inside the UI to augment the LLM dynamically without system reboots.